SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

PRACTICAL - 4

AIM :- To implement the Recursive Best-First Search (RBFS) algorithm to find the best route from Home (Kush Sapphire, Andheri West) to Mantralaya, Mumbai, using the available road network and compare the three available routes based on distance and travel time.

Code:-

```
# T101 RAJDEEP M PARAB
# AIM: Implement Recursive Best-First Search (RBFS)
# Problem: Find the best route from Home (Kush Sapphire)to Mantralaya Mumbai.
import queue as Q
start = 'Home'
goal = 'Mantralaya'
# These values represent the estimated distance from each
# major point to Mantralaya.
dict_hn = {
   'Home': 27.5,
   'Swami Vivekanand Rd': 26.4,
   'Route1_Jamshedji Tata Road': 0.5,
   'Route2_Dadar_Mohammed_Ali_Rd': 1.3,
   'Route3_Dadar_Mohammed_Ali_Rd': 1.3,
   'Mantralaya': 0
}
# Road network with distances in km
# The three route documents provide three alternatives
# road networks from Home to Mantralaya.
# Route 1:
# Home -> Swami Vivekanand Rd
#      -> Jamshedji Tata Road
#      -> Mantralaya
# Route 2:
# Home -> Swami Vivekanand Rd
#      -> Dadar / Mohammed Ali Road
#      -> Mantralaya
# Route 3:
# Home -> Swami Vivekanand Rd
#      -> Dadar / Mohammed Ali Road
#      -> Mantralaya
dict_gn = {
   'Home': {
      'Route1_Start': 1.1,
      'Route2_Start': 1.1,
      'Route3_Start': 1.1
   },
   # ---------- ROUTE 1 ----------
   'Route1_Start': {
      'Route1_Jamshedji Tata Road': 26.6
   },
```

```
    'Route1_Jamshedji Tata Road': {
        'Mantralaya': 0.5
    },
    # ---------- ROUTE 2 ----------
    'Route2_Start': {
        'Route2_Dadar_Mohammed_Ali_Rd': 25.9
    },
    'Route2_Dadar_Mohammed_Ali_Rd': {
        'Mantralaya': 0.5
    },
    # ---------- ROUTE 3 ----------
    'Route3_Start': {
        'Route3_Dadar_Mohammed_Ali_Rd': 25.9
    },

    'Route3_Dadar_Mohammed_Ali_Rd': {
        'Mantralaya': 0.5
    },
    'Mantralaya': {}
}
# Actual distances mentioned in the three Google Maps docs
route_distance = {
    'Route 1': 28.2,
    'Route 2': 27.5,
    'Route 3': 27.5
}
route_time = {
    'Route 1': '58 min',
    'Route 2': '58 min',
    'Route 3': '59 min'
}
# Calculate f(n) = g(n) + h(n)
def get_fn(citystr):
    cities = citystr.split(',')
    gn = 0
    for ctr in range(0, len(cities) - 1):
        gn = gn + dict_gn[cities[ctr]][cities[ctr + 1]]
    hn = dict_hn.get(cities[-1], 0)
    return gn + hn

# Display Priority Queue
def printout(cityq):
    print("\nCurrent Priority Queue:")
    for i in range(0, cityq.qsize()):
        print(cityq.queue[i])

# Recursive Best-First Search
result = ''
def expand(cityq):
    global result
    # Remove node having lowest f(n)
```

```
tot, citystr, thiscity = cityq.get()
# Find second-best f(n)
nexttot = 999999
if not cityq.empty():
    nexttot, nextcitystr, nextthiscity = cityq.queue[0]

# Goal test
if thiscity == goal and tot <= nexttot:
    result = citystr + '::' + str(tot)
    return
print("\nExpanded city ----------",
      thiscity)
print("Current f(n) ----------",
      round(tot, 2))
print("Second best f(n) ----------",
      round(nexttot, 2))
# Temporary queue for successors
tempq = Q.PriorityQueue()
# Expand current node
for cty in dict_gn[thiscity]:
    newcitystr = citystr + ',' + cty
    tempq.put(
        (
            get_fn(newcitystr),
            newcitystr,
            cty
        )
    )
# Add best successors
while not tempq.empty():
    ctrtot, ctrcitystr, ctrthiscity = tempq.get()
    if ctrtot < nexttot:
        cityq.put(
            (
                ctrtot,
                ctrcitystr,
                ctrthiscity
            )
        )
    else:
        cityq.put(
            (
                ctrtot,
                citystr,
                thiscity
            )
        )
        break
printout(cityq)
# Recursive call
expand(cityq)
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

```
# Main RBFS
def main():
    global result
    cityq = Q.PriorityQueue()
    # Dummy node, similar to the Romanian map program
    cityq.put(
        (
            999999,
            "NA",
            "NA"
        )
    )
    # Put starting node into priority queue
    cityq.put(
        (
            get_fn(start),
            start,
            start
        )
    )
    expand(cityq)
    print("\n\nFinal RBFS Result:")
    print(result)

# Run RBFS
main()

# ROUTE COMPARISON
print("\n")
print("=" * 65)
print("INDIVIDUAL ROUTE DISTANCE COMPARISON")
print("=" * 65)
print("\nRoute 1:")
print("Distance :", route_distance['Route 1'], "km")
print("Time     :", route_time['Route 1'])
print("\nRoute 2:")
print("Distance :", route_distance['Route 2'], "km")
print("Time     :", route_time['Route 2'])
print("\nRoute 3:")
print("Distance :", route_distance['Route 3'], "km")
print("Time     :", route_time['Route 3'])

# Find minimum distance
best_distance = min(route_distance.values())
best_routes = []
for route, distance in route_distance.items():
    if distance == best_distance:
        best_routes.append(route)
print("\n")
print("=" * 65)
```

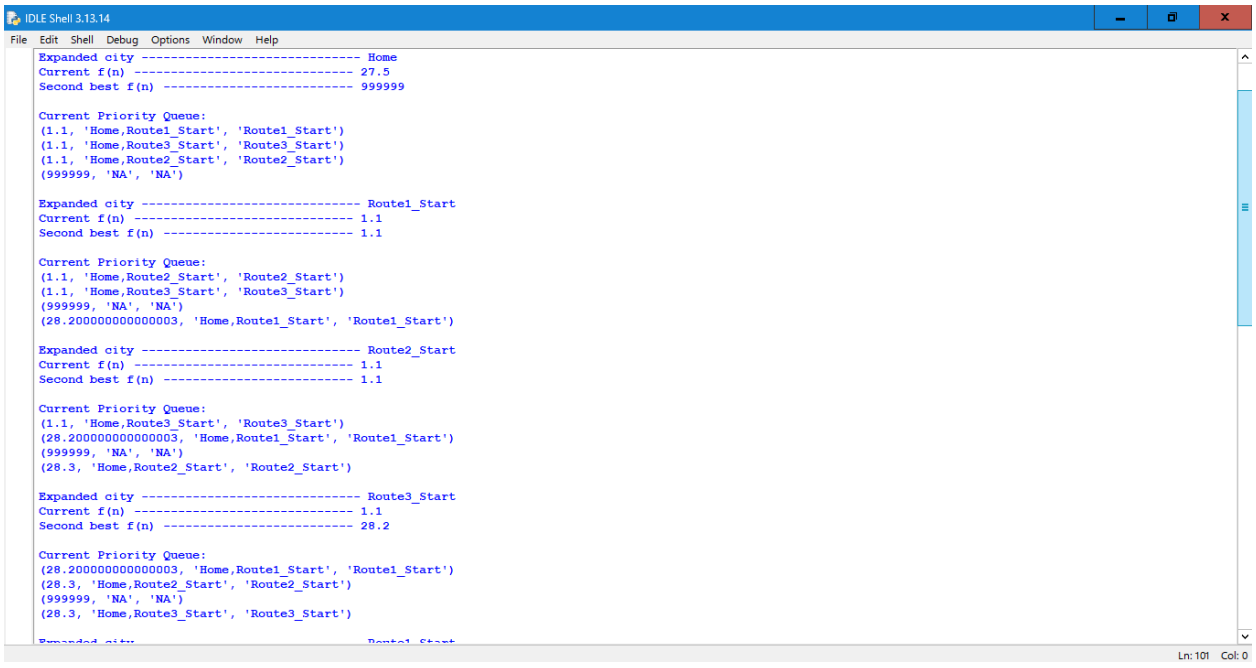
T101
RAJDEEPP M PARAB
TY.BSc.CS

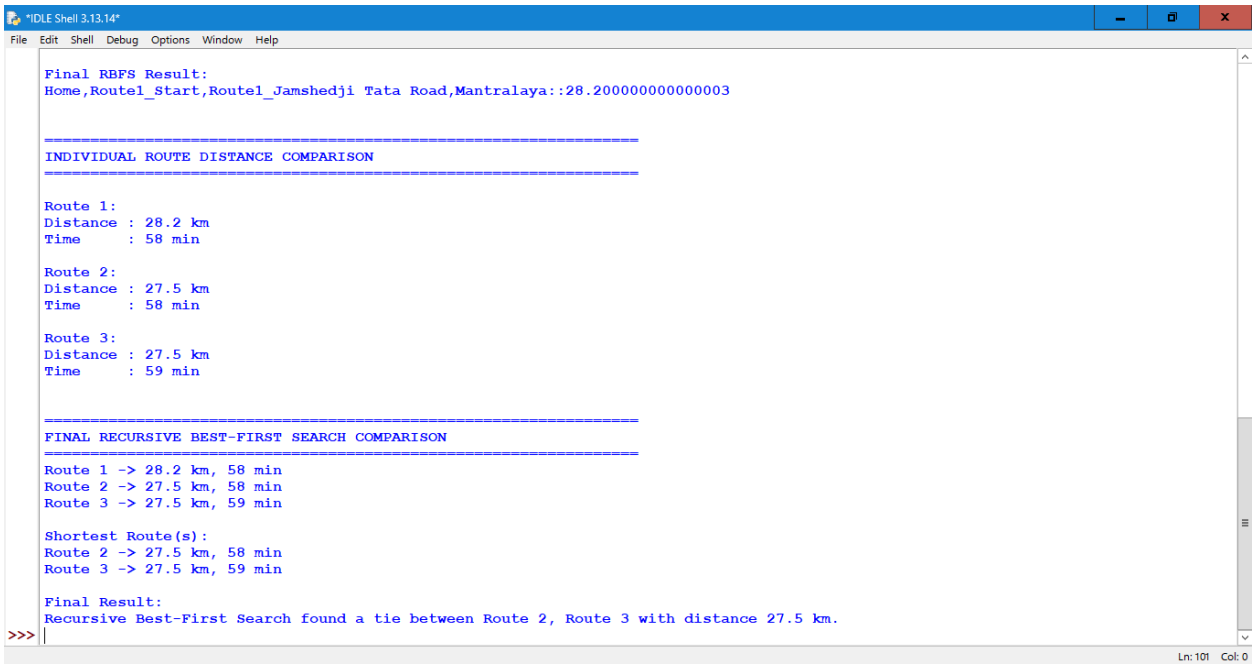
```
print("FINAL RECURSIVE BEST-FIRST SEARCH COMPARISON")
print("=" * 65)
for route in route_distance:
    print(
        route,
        "->",
        route_distance[route],
        "km,",
        route_time[route]
    )
print("\nShortest Route(s):")
for route in best_routes:
    print(
        route,
        "->",
        route_distance[route],
        "km,",
        route_time[route]
    )
print("\nFinal Result:")
if len(best_routes) == 1:
    print(
        "Recursive Best-First Search selected",
        best_routes[0],
        "as the shortest route."
    )
else:
    print(
        "Recursive Best-First Search found a tie between",
        ", ".join(best_routes),
        "with distance",
        best_distance,
        "km."
    )
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

Output:-





T101
RAJDEEPP M PARAB
TY.BSc.CS